# CSS Bundling

Rails manages CSS bundling with the cssbundling-rails gem in much the same way it manages JavaScript bundling—it defers to a third-party tool that it expects to deliver a packaged CSS file to a known location, and then it watches for changed files that trigger a rebundle.

For our project, we are using Tailwind for our bundling, but the cssbundling-rails gem also supports Sass and PostCSS, both of which can also package CSS files. You can also specify some popular frameworks that are based on Sass. Where this gets a little confusing is that all of these tools have slightly different feature sets and goals, while overlapping on some common features.

## Tailwind

Tailwind defines a series of "utility classes," which are small CSS classes that generally are shortcuts for one or two CSS styles set to particular values. For bundling purposes, Tailwind is also a command-line tool that generates a real CSS file based on the utility classes used in your code and merged with a starting file, which we currently store at app/assets/stylesheets/application.tailwind.css. It doesn't do any further post-processing of CSS because in general, when you use Tailwind, you don't write much custom CSS for it to process. The Tailwind classes take care of most styling needs. As we've seen, if you use CSS bundling and Tailwind, you get a Tailwind starter file and a build script that calls the Tailwind CLI.

## Sass

Sass is actually a CSS extension language (the name alleges to be an acronym for Syntactically Awesome Style Sheets), which means it provides syntax that extends CSS to make certain things easier, such as managing nested DOM selectors or variable values. Sass also allows file import, so it came to be used somewhat informally to bundle up CSS (for example, in situations where a startup file was used as a manifest that would import in other files where the actual Sass code would get written).